JavaEE进阶-加餐资料

1. Spring相关

1. **知识点: IoC& AOP**
    a. ioc如何实现的
    b. 项目中用了哪些AOP
    c. 哪些功能是AOP实现的
    d. Spring的核心思想
    e. Spring全局异常处理器原理
    f. 如何在Spring中使用依赖注入
    g. 注入方式有哪些
    h. Spring创建的对象是单例的还是多例的？还是原型的？
    i. Spring哪些特性是基于AOP实现的
    j. Springboot中不想加载一个bean如何做
    k. SpringAOP的机制可以讲一下吗
    l. SpringAOP基于什么实现
    m. Spring怎么注入类
    n. Spring的AOP在哪用过
    o. 多个相同类型的bean，Spring如何判定我们需要的哪一个
    p. Spring中@Resource和@Autowired区别
    q. Spring怎么用到反射的
    r. 自己自定义过注解吗，怎么实现的
    s. Spring AOP 的使用场景
2. **知识点: Spring 全家桶**
    a. Spring和SpringBoot区别
    b. Spring、SpringBoot、SpringCloud区别
    c. Spring和Spring boot的区别
    d. 你怎么理解Spring全家桶
    e. 说说Springboot的好处

f. Spring Boot 是什么
g. Spring、Springmvc、Springboot是什么？ 他们仨的区别？
h. 为什么用Springcloud，微服务有什么好处
i. Springcloud用到了哪些
j. 说一下对Spring Cloud的理解

3. **知识点: 事务相关**
   a. Spring中的事务和mysql的事务区别
   b. Spring中的事务什么时间失效
   c. 怎么解决Spring事务失效
   d. 能不能说说Spring事务
   e. 介绍一下Spring的事务隔离级别
   f. 不加 public 为何事务失效
   g. Spring中方法是private修饰，事务会生效吗
   h. Spring中事务是如何实现的？
   i. Spring中事务的隔离级别？
   j. Spring中事务的传播机制？
   k. Spring中事务失效的场景有哪些？解决方法？
   l. Springboot里面的事务传递类型有什么
4. **知识点: 设计模式**
   a. IoC使用了哪些设计模式
   b. 讲讲反射和动态代理, 代理模式怎么实现的？
   c. Springboot体现的设计模式说一下
   d. Spring框架使用到的设计模式
   e. Spring的代理机制用过哪些，jdk代理的局限性，代理类的创建过程
   f. SpringAOP的机制可以讲一下吗
   g. Spring用了哪些设计模式（单例、代理, 适配器）
   h. 了解哪些设计模式？在Spring中哪些地方用到了什么设计模式
   i. Spring中你知道的设计模式
5. **知识点: 自动配置**
   a. Spring的自动配置怎么做的
   b. 讲讲SpringBoot自动装配

c. 为什么Springboot不需要配置那么多东西？这是通过什么实现的，说下底层原理和过程
d. SpringApplication注解里面是干什么的？
e. Spring Boot 自动装配流程是什么？
f. SpringBoot的自动装配原理
g. Springboot 如何做到启动的时候注入一些bean

6. **知识点: 三级缓存**
   a. Spring怎么解决循环依赖/循环引用
   b. Spring源码看过吗？Spring的三级缓存知道吗？
   c. Spring三级缓存为什么要设置第三级
   d. Spring3.6以前是如何解决循环依赖问题的
7. **知识点: 生命周期**
   a. Spring里一个bean的生命周期
   b. Spring Bean的初始化过程
   c. Spring框架中的单例Bean是线程安全的吗
   d. SpringBean的生命周期有哪些
8. **知识点: 注解相关**
   a. Springboot如何接收前端发来的JSON数据
   b. Springboot常用注解
   c. Springboot中如何定义post方法并接收json参数
   d. Springboot中写一个http接口的流程
   e. SpringBoot如何读取配置文件的内容
9. **源码**
   a. Spring源码有研究过吗？
   b. 讲一下你了解的Spring源码
   c. 阅读过Spring源码吗
10. **其他**
   a. Spring的数据库连接池
   b. SpringMVC的执行流程了解吗
   c. Springboot中开发环境和测试环境的配置文件是怎么区分的
   d. Spring 里面单独提出了一个 Bean 的概念，和 java 里面的bean 有什么区别

2. Spring三级缓存的理解

这个问题或者换个问法：Spring是如何解决循环依赖的？答案即是Spring的三级缓存

什么是循环依赖

简单说，就是A对象依赖B对象，B对象又依赖A对象，类似的代码如下：

```
@Component
public class A{
    @Autowired
    private B b;
}
@Component
public class B{
    @Autowired
    private A a;
}
```

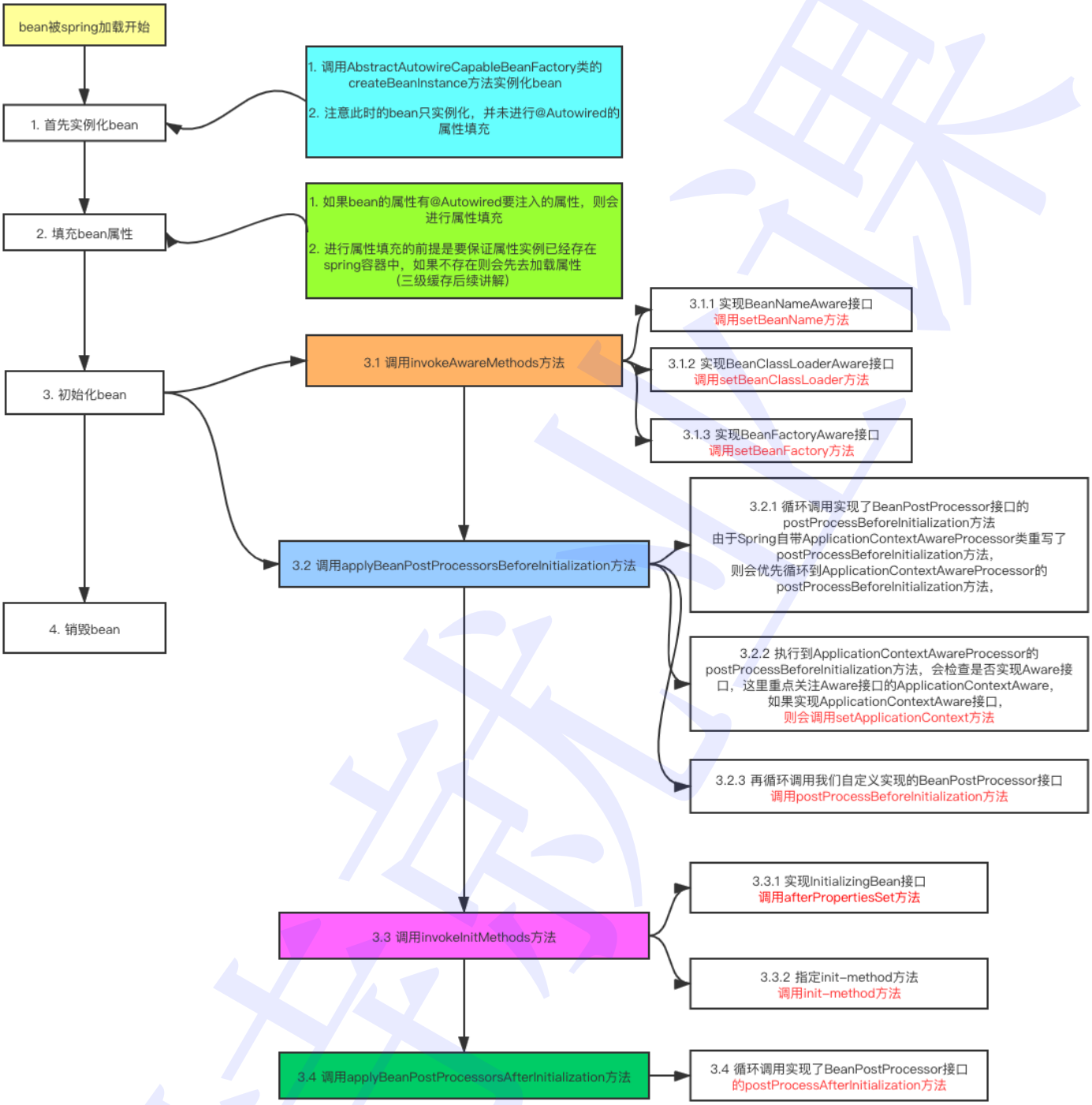
其他还有很多种方式，如A依赖B，B依赖C，C依赖A，或是A依赖A自己，只要产生了依赖关系的闭环，即造成了循环依赖。

那么，循环依赖会引发什么问题呢？理解这个问题先得理解Bean的生命周期，以下先回顾下

Bean的生命周期回顾

1. 启动容器：加载Bean
2. 实例化Bean对象
3. 依赖注入：装配Bean的属性
4. 初始化Bean：执行aware接口方法、预初始化方法、初始化方法、后初始化方法
5. 关闭容器：销毁Bean

在以上第四个步骤执行完毕，才算一个初始化完成的Bean，也即Spring容器中完整的Bean对象。



循环依赖的问题

Spring容器保存Bean的方式，是采取缓存的方式：使用 `Map<String, Object>` 的结构，key为Bean的名称，value为Bean对象。需要使用时直接从缓存获取。

假如A、B互相依赖（循环依赖）：

1. 容器中没有A对象，实例化A对象
2. 装配A中的B对象，发现B在容器中没有，需要先实例化B
3. 实例化B对象
4. 装配B中的A对象，发现A在容器中没有，需要先实例化A
5. 重复第一个步骤

这就套娃了, 你猜是先 `StackOverflow` 还是 `OutOfMemory` ?

[PS]

- Bean会依赖某些注入的Bean来完成初始化工作
- 由于Spring支持构造方法注入，属性/Setter注入的方式，所以不能简单的先把所有对象全部实例化，放到缓存中，再全部执行初始化。原因很简单，此时所有对象的引用都可以获取到，但属性都是null，执行初始化甚至构造方法都可能出现空指针异常。

那么我们说Spring能解决循环依赖，也不是所有的情况都可以解决，只有以下情况才支持。

Spring不支持的循环依赖

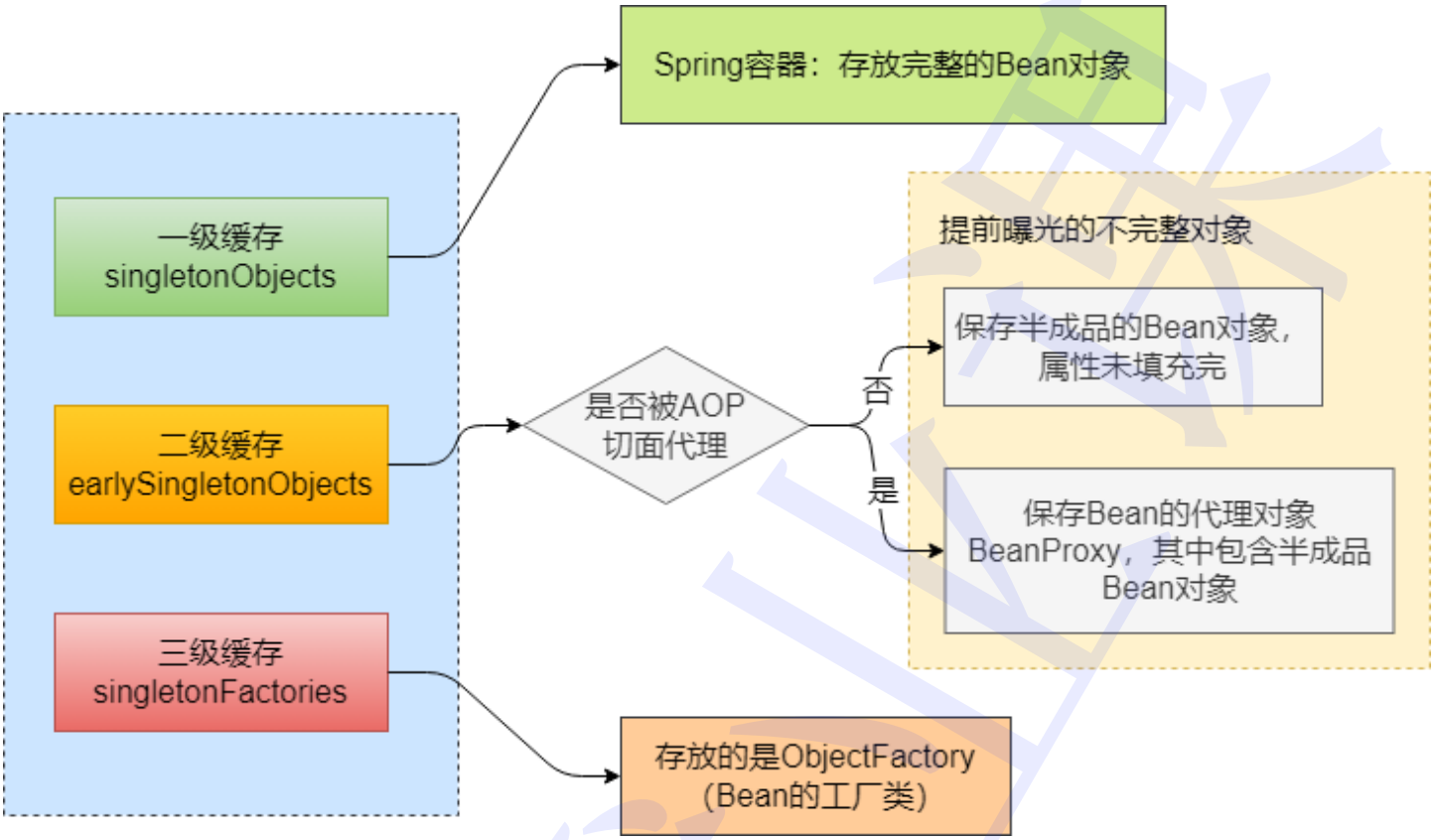
Spring框架在处理循环依赖时存在一些限制, 它不能解决所有类型的循环依赖问题

比如以下循环依赖的场景, Spring并不支持(包含但不限于)

1. **构造器循环依赖**：当两个Bean在它们的构造器中相互依赖时，Spring无法解决这种循环依赖。Spring在创建Bean时会检查构造器中的循环依赖，并在发现时抛出 `BeanCurrentlyInCreationException` 异常。
2. **原型Bean循环依赖**：对于原型作用域（Prototype）的Bean，Spring不支持循环依赖。原型Bean的生命周期是短暂的，每次请求都会创建新实例，Spring容器不缓存原型Bean，因此无法提前暴露一个创建中的Bean来解决循环。
3. **复杂的AOP场景下的循环依赖**：在一些复杂的AOP（面向切面编程）场景中，如果Bean之间存在循环依赖，并且这些Bean被用于AOP代理，Spring可能无法解决这种循环依赖。

Spring解决循环依赖

Spring是使用三级缓存的机制来解决循环依赖问题，以下为三级缓存的定义：



三级缓存的源码见 `DefaultSingletonBeanRegistry`：

```
public class DefaultSingletonBeanRegistry extends SimpleAliasRegistry implements SingletonBeanRegistry {
        /** Cache of singleton objects: bean name to bean instance. */
        private final Map<String, Object> singletonObjects = new ConcurrentHashMap<>(256);//一级缓存
        /** Cache of singleton factories: bean name to ObjectFactory. */
        private final Map<String, ObjectFactory<?>> singletonFactories = new HashMap<>(16);//三级缓存
        /** Cache of early singleton objects: bean name to bean instance. */
        private final Map<String, Object> earlySingletonObjects = new ConcurrentHashMap<>(16);//二级缓存
}
```

以下是部分说明：

- 三级缓存singletonFactories中保存的是ObjectFactory对象（Bean工厂），其中包含了BeanName，Bean对象，RootBeanDefinition，该工厂可以生成Bean对象。
- 由于Bean可能被代理，此时注入到其他Bean属性中的也应该是代理Bean。

单例模式的A、B循环依赖执行流程如下：

步骤	操作	三层列表中的内容
1	开始初始化对象A：创建A对象，此时只是new了，field还没设置	singletonFactories： earlySingletonObjects： singletonObjects：
2	添加到第三级缓存：加进去的，只是个factory，只有循环依赖的时候，才会发挥作用	singletonFactories：A earlySingletonObjects： singletonObjects：
3	注入A对象属性：B。此时一、二、三级缓存都获取不到B，需要先初始化B	singletonFactories：A earlySingletonObjects： singletonObjects：
4	开始初始化对象B：创建B对象（new），并添加到第三级缓存	singletonFactories：A, B earlySingletonObjects： singletonObjects：
5	注入B对象属性：A。此时发现存在循环依赖（原因涉及更多细节，不是此处关注重点，略），在第三级缓存中找到A的ObjectFactory，调用方法获取Bean，并将Bean存放到二级缓存，及在第三级缓存中删除A的ObjectFactory。获取到的Bean可能是代理类，没有被代理则获取原始Bean	singletonFactories：B earlySingletonObjects：A singletonObjects：
6	B继续后续的处理：注入其他依赖、初始化。处理完成后，B为完整的Bean，把B放入一级缓存，并在二级缓存和三级缓存中删除	singletonFactories： earlySingletonObjects：A singletonObjects：B
7	方法调用链返回：即第3步执行完，A对象成功注入属性B。继续注入A的其他依赖，及初始化。处理完成后，A为完整的Bean，把A放入一级缓存，并在二级缓存和三级缓存中删除。	singletonFactories： earlySingletonObjects： singletonObjects：A, B

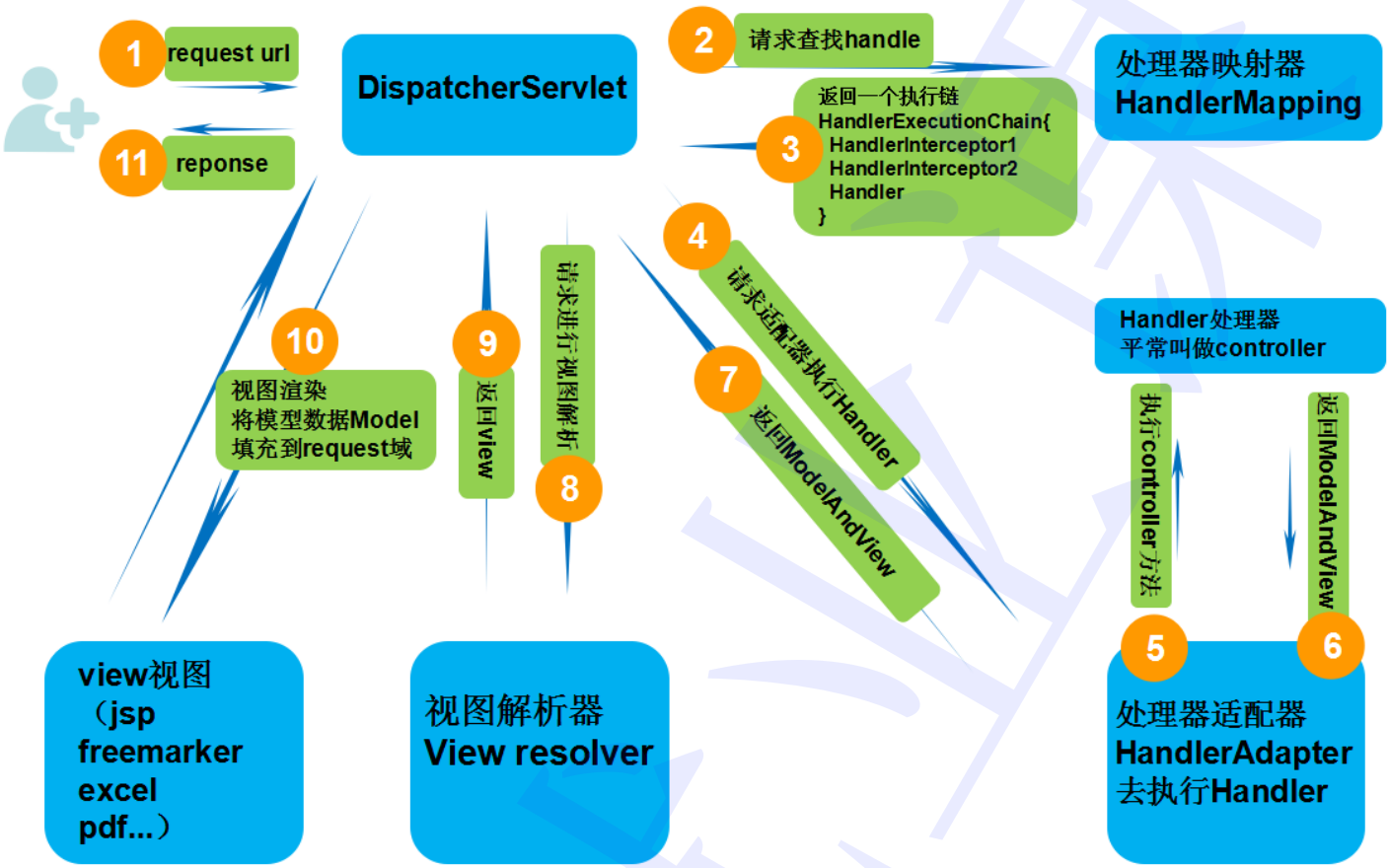
为什么要使用三级缓存

依照以上三级缓存的流程，其实使用二级缓存也能满足循环依赖的注入：

- 普通的IoC容器使用一级缓存即可，但无法解决循环依赖问题。
- 解决循环依赖问题：使用二级缓存即可。一级缓存保存完整Bean，二级缓存保存提前曝光的不完整的Bean。
- 需要AOP代理Bean时，有两种实现思路：
  （1）再加一级缓存
  （2）只使用二级缓存，其中二级缓存保存Bean的代理对象，代理对象中引用不完整的原始对象即可
- Spring使用三级缓存保存ObjectFactory即Bean工厂，在代码的层次设计及扩展性上都会更好。三级缓存的引入主要是为了解决代理相关的循环依赖问题，尤其是涉及到AOP时，代理对象的创建可能需要提前于Bean的完全初始化.

> 💡 在Spring 3.6之前，Spring通过使用一级和二级缓存来解决单例Bean的循环依赖问题，而从3.6版本开始，通过引入三级缓存，Spring能够更全面地处理包括AOP在内的更复杂的循环依赖情况。

3. SpringMVC的流程



SpringMVC的请求响应步骤如下：

具体步骤：

- 第一步：（**发起**）发起请求到前端控制器(DispatcherServlet)
- 第二步：（**查找**）前端控制器请求HandlerMapping查找 Handler（可以根据xml配置、注解进行查找）
- 第三步：（**返回**）处理器映射器HandlerMapping向前端控制器返回Handler，HandlerMapping会把请求映射为HandlerExecutionChain对象（包含一个Handler处理器（页面控制器）对象，多个HandlerInterceptor拦截器对象），通过这种策略模式，很容易添加新的映射策略
- 第四步：（**调用**）前端控制器调用处理器适配器去执行Handler
- 第五步：（**执行**）处理器适配器HandlerAdapter将会根据适配的结果去执行Handler
- 第六步：（**返回**）Handler执行完成给适配器返回ModelAndView
- 第七步：（**接收**）处理器适配器向前端控制器返回ModelAndView （ModelAndView是SpringMVC框架的一个底层对象，包括 Model和view）
- 第八步：（**解析**）前端控制器请求视图解析器去进行视图解析 （根据逻辑视图名解析成真正的视图(jsp)），通过这种策略很容易更换其他视图技术，只需要更改视图解析器即可
- 第九步：（**返回**）视图解析器向前端控制器返回View

- 第十步：（**渲染**）前端控制器进行视图渲染 （视图渲染将模型数据(在ModelAndView对象中)填充到request域）
- 第十一步：（**响应**）前端控制器向用户响应结果

> 以下是对出现的一些组件的介绍：
>
> (1) DispatcherServlet
>
> 作用：接收请求，响应结果，相当于转发器，中央处理器。有了DispatcherServlet减少了其它组件之间的耦合度。
>
> (2) 处理器映射器HandlerMapping
>
> 作用：根据请求的url查找Handler
>
> (3) 处理器适配器HandlerAdapter
>
> 作用：按照特定规则（HandlerAdapter要求的规则）去执行Handler。
>
> (4) 处理器Handler(程序员开发)
>
> 注意：编写Handler时按照HandlerAdapter的要求去做，这样适配器才可以去正确执行Handler
>
> (5) 视图解析器ViewResolver
>
> 作用：进行视图解析，根据逻辑视图名解析成真正的视图
>
> (6) 视图View(程序员开发)
>
> 注意：View是一个接口，实现类支持不同的View类型（jsp、freemarker、pdf...）
>
> ps:不需要程序员开发的，需要程序员自己做一下配置即可

比特就业课